

ChatDoc AI

Agente de Consulta Documental Basado en RAG

Informe Técnico

Arquitectura de Agentes Inteligentes

Identificación de Sensores, Decisiones y Actuadores

Mayo 2026

Índice

Resumen

El presente informe describe el diseño, implementación y comportamiento del sistema **ChatDoc AI**, un agente inteligente de consulta documental que permite a los usuarios cargar documentos (PDF, DOCX, TXT) y realizar preguntas en lenguaje natural sobre su contenido. El agente aplica el paradigma *Retrieval-Augmented Generation* (RAG), combinando búsqueda semántica vectorial con generación de texto mediante un modelo de lenguaje de gran escala (LLM).

La arquitectura se apoya en **LlamaIndex** como framework de orquestación, **ChromaDB** como base de datos vectorial persistente, **HuggingFace embeddings** (BAAI/bge-small-en-v1.5) para la vectorización y **Groq** con el modelo **llama-3.3-70b-versatile** como motor de respuesta. El frontend, desarrollado en React con Vite y Tailwind CSS, ofrece una interfaz de usuario moderna con soporte de tema claro/oscuro.

Se identifican y analizan los **sensores**, **módulos de decisión** y **actuadores** del agente, mostrando cómo cada componente participa en el ciclo percepción–razonamiento–acción propio de los sistemas basados en agentes.

Palabras clave: RAG, LLM, agente inteligente, LlamaIndex, ChromaDB, FastAPI, React.

Introducción

En los últimos años, los modelos de lenguaje de gran escala (LLMs) han demostrado capacidades notables para la comprensión y generación de texto. Sin embargo, su entrenamiento estático no contempla documentación privada o especializada propia de cada organización. El patrón **RAG** (*Retrieval-Augmented Generation*) emerge como solución: combina la recuperación semántica de fragmentos relevantes de documentos con la capacidad generativa del LLM, restringiendo las respuestas al contexto cargado.

ChatDoc AI es un sistema RAG completo que implementa este paradigma con las siguientes características:

- Ingesta y vectorización de documentos en tiempo real (PDF, DOCX, TXT).
- Almacenamiento persistente de vectores con metadatos en ChromaDB.
- Consulta en lenguaje natural con respuestas fundamentadas en documentos.
- Filtrado opcional por documento específico para mayor precisión.
- Interfaz gráfica moderna con notificaciones de estado.

Desde la perspectiva de los agentes inteligentes, el sistema puede modelarse como un **agente reactivo informado**, donde los sensores reciben la entrada del entorno (usuario y documentos), el módulo de decisión razona sobre esa información mediante búsqueda semántica y el LLM, y los actuadores producen respuestas y modificaciones en el estado del sistema.

Objetivos

Objetivo General

Diseñar e implementar un agente inteligente de consulta documental que permita a los usuarios interactuar mediante lenguaje natural con el contenido de sus propios documentos, garantizando respuestas contextualmente precisas y limitadas al corpus cargado.

Objetivos Específicos

1. Implementar un *pipeline* de ingesta documental que procese, fragmente y vectorice documentos en formato PDF, DOCX y TXT.
2. Configurar y mantener una base de datos vectorial (ChromaDB) con persistencia en disco para almacenar los *embeddings* y metadatos de los documentos.
3. Integrar un LLM de alta capacidad (Llama 3.3-70b via Groq) para la generación de respuestas basadas exclusivamente en el contexto recuperado.
4. Desarrollar una API REST (FastAPI) que exponga los servicios de ingesta y consulta al frontend.
5. Construir una interfaz de usuario (React + Tailwind CSS) que permita la carga de documentos, selección de fuentes y conversación con el agente.
6. Identificar y documentar formalmente los **sensores**, **módulos de decisión** y **actuadores** del agente para facilitar su análisis y extensión futura.

Metodología

El desarrollo del sistema siguió una metodología **iterativa e incremental**, organizada en las siguientes fases:

1. **Análisis de requerimientos:** Identificación de casos de uso principales (carga de documentos, consulta en lenguaje natural, filtrado por fuente) y restricciones técnicas (respuestas basadas exclusivamente en el contexto).
2. **Diseño de arquitectura:** Selección del stack tecnológico (LlamaIndex, ChromaDB, Groq, FastAPI, React) y definición del flujo de datos entre componentes.
3. **Implementación del backend:** Desarrollo de los servicios de ingesta (`ingesta.py`) y consulta (`chat.py`), con exposición via FastAPI.
4. **Implementación del frontend:** Desarrollo de los componentes React (Sidebar, ChatWindow, MessageBubble, Header) y la capa de comunicación API con Axios.
5. **Pruebas de integración:** Verificación del pipeline completo mediante el archivo `test_chat.py` y el archivo de peticiones HTTP `requests.http`.

6. **Análisis de comportamiento del agente:** Identificación y clasificación de sensores, decisiones y actuadores bajo el marco teórico de agentes inteligentes.

Desarrollo

Arquitectura General del Sistema

El sistema se estructura en dos capas principales: un **backend** Python (FastAPI) y un **frontend** JavaScript (React/Vite). Ambas capas se comunican mediante una API REST con CORS habilitado.

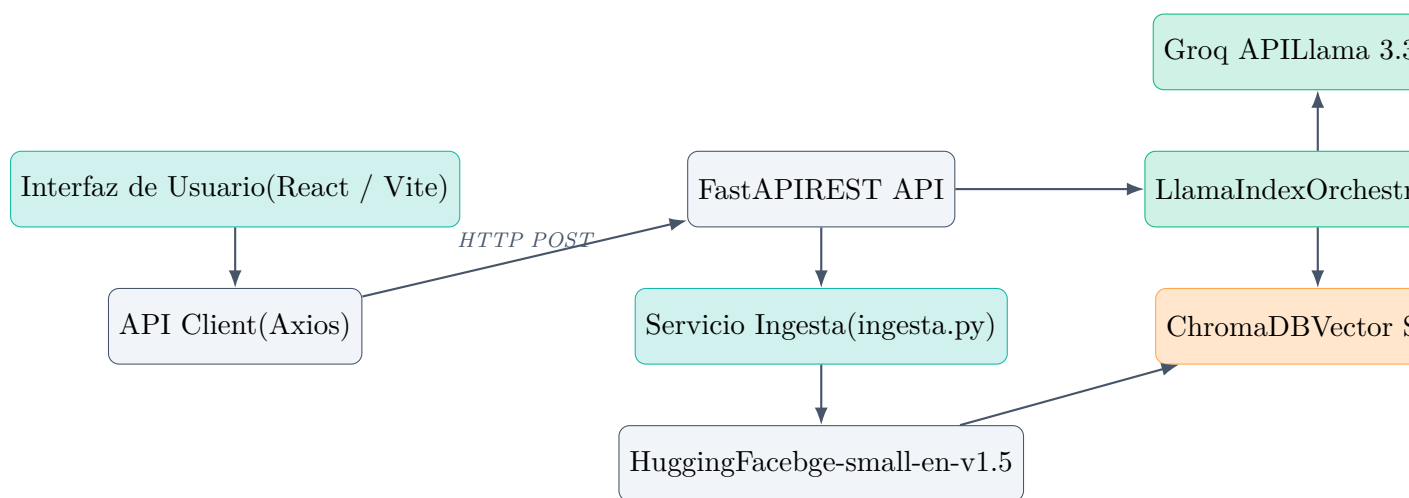


Figura 1: Diagrama de arquitectura general del sistema ChatDoc AI.

Identificación de Sensores, Decisiones y Actuadores

La siguiente tabla resume de forma concisa el rol de cada componente dentro del ciclo del agente:

Categoría	Componente	Descripción	Módulo
teal!10 Sensor	Entrada de texto	Pregunta del usuario en lenguaje natural	ChatWindow.jsx
teal!10 Sensor	Carga de archivo	Documento cargado (PDF/DOCX/TXT) via drag-and-drop	Sidebar.jsx
teal!10 Sensor	Selección de fuente	Archivos activos para filtrar la consulta	Sidebar.jsx
teal!10 Sensor	Estado HTTP	Respuesta del servidor (éxito/error de conexión)	api.js
emerald!10 Decisión	Vectorización	Conversión de texto a embeddings semánticos	ingesta.py
emerald!10 Decisión	Indexación	Almacenamiento vectorial con metadatos en ChromaDB	ingesta.py
emerald!10 Decisión	Recuperación semántica	Búsqueda por similitud del vector de la pregunta	chat.py
emerald!10 Decisión	Filtrado por fuente	Aplicación de MetadataFilters por nombre de archivo	chat.py
emerald!10 Decisión	Generación RAG	Síntesis de respuesta con LLM + contexto recuperado	chat.py
emerald!10 Decisión	Prompt restrictivo	Política de respuesta: solo contexto, sin inventar	chat.py
orange!10 Actuador	Respuesta en chat	Texto de respuesta mostrado en la interfaz	ChatWindow.jsx
orange!10 Actuador	Notificación toast	Alerta visual de éxito o error de ingesta	Sidebar.jsx
orange!10 Actuador	Escritura en ChromaDB	Persistencia de vectores y documentos indexados	ingesta.py
orange!10 Actuador	Indicador de carga	Animación visual mientras el agente procesa	ChatWindow.jsx

Cuadro 1: Clasificación de sensores, decisiones y actuadores del agente ChatDoc AI.

Sensores del Agente

Los sensores son los mecanismos mediante los cuales el agente **percibe el entorno**. En ChatDoc AI se identifican cuatro sensores principales:

S1 – Entrada de Texto (Pregunta del Usuario)

El componente `ChatWindow.jsx` contiene un área de texto (`<textarea>`) que captura la pregunta del usuario. La acción se dispara al presionar **Enter** (sin Shift) o el botón de envío. El texto es validado (no vacío, no en estado de carga) antes de enviarse.

S2 – Carga de Documento

El componente `Sidebar.jsx` implementa una zona de *drag-and-drop* mediante la librería `react-dropzone`. Acepta los tipos MIME: `application/pdf`, `text/plain`, `application/vnd.openxmlformats-officedocument.wordprocessingml.document` y `application/msword`. Cada archivo se envía al endpoint `POST /ingestar/` del backend.

S3 – Selección de Fuente Activa

El usuario puede marcar/desmarcar documentos en la lista de fuentes del sidebar. Esta selección determina si la consulta se aplica a **todos los documentos** o solo a los **archivos seleccionados**, afectando directamente al filtrado semántico.

S4 – Estado de la Respuesta HTTP

La capa de API (`api.js`) utiliza `Axios` para comunicarse con el backend. El resultado de las llamadas (éxito o error HTTP) es interpretado por el frontend para actualizar el estado de la interfaz y mostrar notificaciones adecuadas.

Módulos de Decisión del Agente

Los módulos de decisión implementan el **razonamiento** del agente: cómo procesa la percepción para determinar una acción.

D1 – Vectorización con HuggingFace Embeddings

Durante la ingesta, cada fragmento del documento es convertido a un vector numérico de alta dimensión mediante el modelo `BAAI/bge-small-en-v1.5`. Este modelo de código abierto, ejecutado localmente, captura el significado semántico del texto.

Listing 1: Configuración del modelo de embeddings.

```
1 Settings.embed_model = HuggingFaceEmbedding(  
2     model_name="BAAI/bge-small-en-v1.5"  
3 )
```

D2 – Indexación en ChromaDB

Los vectores generados son almacenados en una colección persistente de ChromaDB (`documentos_usuario`), junto con los metadatos del documento (nombre de archivo, fragmento de texto). Este índice persiste en disco en `./data/chroma_db`.

D3 – Recuperación Semántica (Similarity Search)

Al recibir una pregunta, el agente vectoriza la consulta con el mismo modelo de embeddings y realiza una **búsqueda por similitud del coseno** en ChromaDB, recuperando los fragmentos más relevantes del corpus.

D4 – Filtrado por Metadatos

Si el usuario ha seleccionado uno o más documentos específicos, se aplican `MetadataFilters` con condición `OR` sobre el campo `file_name`, restringiendo la búsqueda semántica solo a los archivos indicados.

Listing 2: Aplicación de filtros por archivo en la consulta.

```
1 filtros = [  
2     MetadataFilter(key="file_name", value=archivo,  
3                   operator=FilterOperator.EQ)  
4     for archivo in archivos  
5 ]  
6 filters = MetadataFilters(  
7     filters=filtros, condition=FilterCondition.OR  
8 )
```

D5 – Generación de Respuesta (RAG con Groq/Llama 3.3)

Los fragmentos recuperados se ensamblan como contexto y se envían junto con la pregunta al modelo `llama-3.3-70b-versatile` via la API de Groq. El LLM genera la respuesta final.

D6 – Prompt Restrictivo (Control de Alucinaciones)

Un *system prompt* estricto instruye al modelo a responder **únicamente** con información del contexto proporcionado. Si la respuesta no está disponible en los documentos, el agente responde: *“Lo siento, esa información no se encuentra en los documentos proporcionados.”*

Actuadores del Agente

Los actuadores son los mecanismos por los cuales el agente **modifica el entorno**.

A1 – Respuesta en Lenguaje Natural

La respuesta generada por el LLM es recibida por el frontend y renderizada en el componente `MessageBubble.jsx` con soporte Markdown, diferenciando visualmente mensajes del usuario (derecha) y del asistente (izquierda).

A2 – Notificaciones Toast

El sistema emite notificaciones visuales no intrusivas mediante la librería `react-hot-toast`: notificación de éxito al completar la ingesta de un documento, y notificación de error en caso de fallo.

A3 – Escritura en Base de Datos Vectorial

La acción más relevante desde el punto de vista del estado interno del agente: la función `procesar_y_almacenar()` escribe los vectores e índices en ChromaDB de forma persistente, ampliando el conocimiento del agente con cada nuevo documento cargado.

A4 – Indicador Visual de Procesamiento

Durante la espera de la respuesta, el componente `ChatWindow.jsx` muestra una animación de tres puntos pulsantes que informa al usuario que el agente está procesando su consulta.

Ciclo del Agente: Diagrama de Flujo

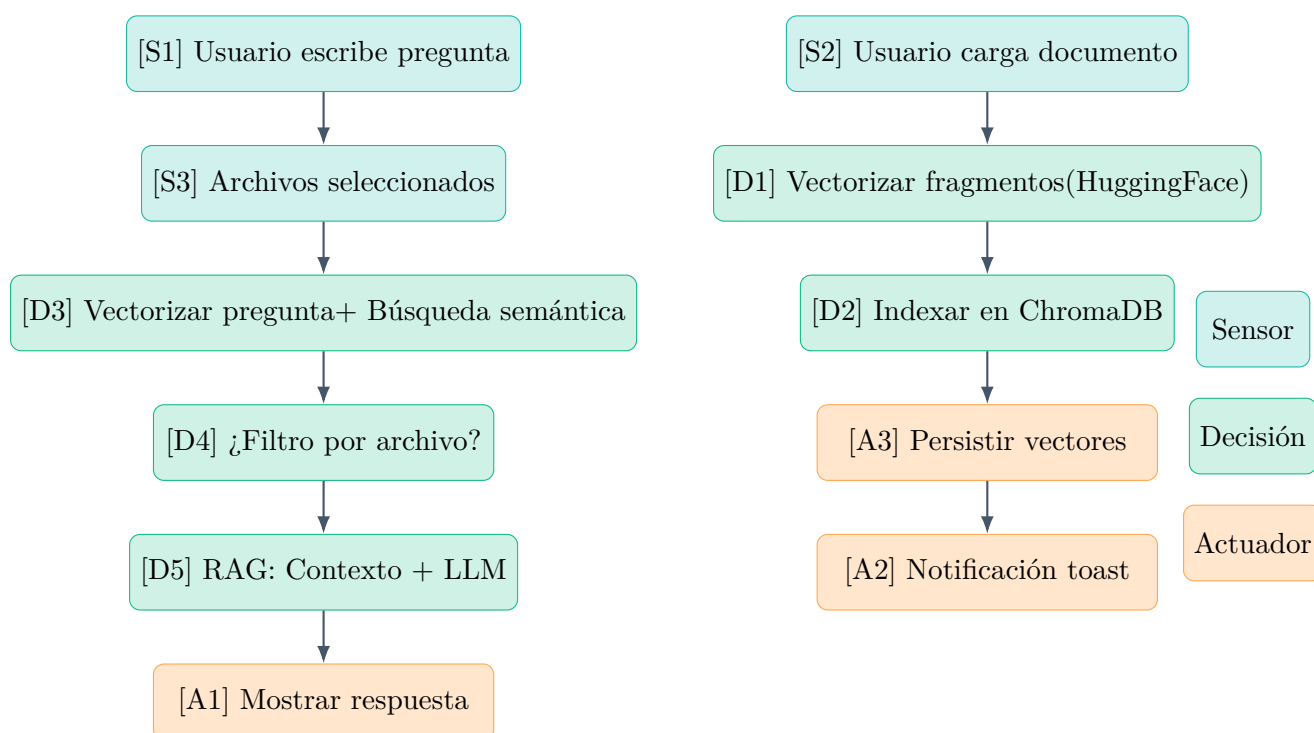


Figura 2: Flujo de percepción-razonamiento-acción del agente ChatDoc AI.

Stack Tecnológico

Capa	Tecnología	Versión/Variante	Rol
Frontend	React	18+	UI / Estado
Frontend	Vite	5+	Bundler / Dev server
Frontend	Tailwind CSS	3	Estilos
Frontend	Framer Motion	–	Animaciones
Frontend	Axios	–	Cliente HTTP
Backend	Python	3.10+	Lenguaje principal
Backend	FastAPI	–	API REST
Backend	LlamaIndex	–	Orquestación RAG
Backend	ChromaDB	–	Base de datos vectorial
Backend	HuggingFace	bge-small-en-v1.5	Embeddings
Backend	Groq API	llama-3.3-70b-versatile	LLM
Backend	PyMuPDF	–	Extracción PDF

Cuadro 2: Stack tecnológico completo de ChatDoc AI.

Resultados

Comportamiento del Agente

El agente ChatDoc AI exhibe el siguiente comportamiento observable durante su operación:

1. **Fase de ingesta:** Al cargar un documento, el agente lo fragmenta automáticamente, genera *embeddings* para cada fragmento y los persiste en ChromaDB. El proceso es transparente para el usuario, quien recibe una notificación visual de éxito o fallo. El backend confirma el número de documentos procesados en la respuesta JSON.
2. **Fase de consulta:** Al formular una pregunta, el agente:
 - a. Vectoriza la pregunta con el mismo modelo de embeddings.
 - b. Recupera los fragmentos más similares del índice (con o sin filtro de archivo).
 - c. Construye un prompt enriquecido con el contexto recuperado.
 - d. Genera la respuesta con el LLM y la devuelve al frontend.
3. **Comportamiento ante preguntas fuera de contexto:** Cuando la pregunta no puede responderse con los documentos cargados, el agente responde con el mensaje predefinido: *“Lo siento, esa información no se encuentra en los documentos proporcionados.”*, evitando alucinaciones.
4. **Filtrado por fuente:** Cuando el usuario selecciona archivos específicos en el sidebar, el agente restringe la búsqueda semántica únicamente a esos documentos, devolviendo respuestas más precisas y acotadas.

Identificación Final: Tabla Consolidada SDA

ID	Tipo	Nombre	Función
S1	Sensor	Entrada de texto	Captura la pregunta del usuario vía teclado o botón
S2	Sensor	Carga de archivo	Recibe documentos por drag-and-drop o selector
S3	Sensor	Selección de fuente	Indica qué archivos usar como contexto de búsqueda
S4	Sensor	Estado HTTP	Detecta éxito o fallo en la comunicación con el backend
D1	Decisión	Vectorización	Transforma texto en vectores semánticos (HuggingFace)
D2	Decisión	Indexación	Almacena vectores y metadatos en ChromaDB
D3	Decisión	Recuperación semántica	Busca fragmentos relevantes por similitud de coseno

ID	Tipo	Nombre	Función
D4	Decisión	Filtrado por meta-datos	Restringe la búsqueda por nombre de archivo
D5	Decisión	Generación RAG	Produce respuesta con Groq/Llama 3.3 y contexto
D6	Decisión	Prompt restrictivo	Previene alucinaciones; responde solo del corpus
A1	Actuador	Respuesta en chat	Muestra la respuesta generada en la interfaz
A2	Actuador	Notificación toast	Informa sobre el resultado de la ingesta
A3	Actuador	Escritura en ChromaDB	Persiste los vectores del nuevo documento
A4	Actuador	Indicador de carga	Muestra animación mientras el agente procesa

Cuadro 3: Tabla consolidada de Sensores, Decisiones y Actuadores del agente ChatDoc AI.

Conclusiones

1. ChatDoc AI implementa exitosamente el paradigma RAG, logrando un agente capaz de responder preguntas en lenguaje natural sobre documentos propios del usuario, sin recurrir a conocimiento externo al corpus proporcionado.
2. El análisis de sensores, decisiones y actuadores confirma que el sistema opera como un **agente reactivo informado**: sus sensores perciben tanto el input del usuario (pregunta, archivo) como el estado del sistema (selección activa, estado HTTP); sus módulos de decisión aplican razonamiento semántico (embeddings, recuperación, filtrado, generación) y sus actuadores modifican el entorno de forma útil (respuesta, persistencia, notificación).
3. La combinación de embeddings locales (HuggingFace) con un LLM en la nube (Groq) representa un balance efectivo entre costo computacional y calidad de respuesta.
4. El uso de ChromaDB con persistencia garantiza que el conocimiento indexado no se pierde entre sesiones, permitiendo un crecimiento progresivo de la base de conocimiento del agente.
5. El prompt restrictivo implementado como D6 es crítico para la confiabilidad del sistema: elimina el riesgo de que el agente genere respuestas falsas basadas en su conocimiento paramétrico.

Recomendaciones

1. **Autenticación y control de acceso:** Implementar un sistema de autenticación (JWT, OAuth2) para aislar los documentos por usuario, especialmente si el sistema se despliega en un entorno multiusuario.
2. **Gestión de colecciones por usuario:** Crear colecciones ChromaDB independientes por usuario o sesión, evitando que las preguntas de un usuario accedan a documentos de otro.
3. **Eliminación de documentos:** Agregar la funcionalidad de eliminar documentos del índice, incluyendo la remoción de sus vectores en ChromaDB (actualmente no existe un endpoint para esta operación).
4. **Chunking configurable:** Exponer parámetros de fragmentación (tamaño de chunk, overlap) como opciones de configuración, ya que tienen impacto directo en la calidad de la recuperación semántica.
5. **Memoria conversacional:** Integrar memoria de contexto conversacional (historial de turnos) en el módulo de decisión D5 para que el agente pueda responder preguntas de seguimiento sin perder el hilo de la conversación.
6. **Soporte de imágenes y tablas en PDF:** Mejorar el extractor de PDF (actualmente PyMuPDF) con capacidades multimodales para indexar contenido visual dentro de los documentos.
7. **Evaluación de calidad RAG:** Incorporar métricas de evaluación (fidelidad, relevancia de recuperación) usando frameworks como RAGAS para monitorear la calidad del agente de forma continua.
8. **Despliegue con Docker:** Containerizar el backend y el frontend con Docker Compose para facilitar el despliegue reproducible en cualquier entorno, incluyendo la correcta persistencia del volumen de ChromaDB.